

Ejercicio 4 Sistemas Distribuidos



Autores:

- **Pablo Pérez Tenorio (100522321)**
 - **Pablo Pino Castillo (100522129)**
-

Ejercicio 4 Sistemas Distribuidos

1. Entidades	1
2. Protocolo de Red	2
3. Servidores — Hilos y Estado	2
4. Mensajes	3
5. Flujo de Mensajes	4
6. Nombrado (IP y Puertos)	5
7. Seguridad	5

1. Entidades

Lo primero a realizar al leer el enunciado proporcionado sería tratar de identificar quiénes participan en el sistema. Este sistema está compuesto por tres tipos de entidades que se comunican entre sí:

- **Computador de caja (×10):** Cada caja dispone de un computador que registra la actividad local de dicha caja (aperturas, ventas y cierre) y la comunica al servidor central. Actúa principalmente como cliente, pero luego veremos que también escucha consultas del servidor principal, por lo que también hace de servidor.
 - **Computador central (servidor):** Recibe y registra los eventos de todas las cajas, actuando como un servidor permanente, y manteniendo el estado global del supermercado. Desde él, el operador se encarga de monitorizar cualquier caja en cualquier momento.
-

2. Protocolo de Red

Transporte: TCP. Los mensajes contienen información que no puede perderse ni llegar desordenada. TCP garantiza entrega fiable, orden y detección de errores sin necesidad de mecanismos adicionales a nivel de aplicación. El volumen de datos es reducido (10 cajas), por lo que su overhead es irrelevante.

Formato: protocolo de texto. Decidimos que los campos se separan con '|' y cada mensaje termina en '\n'. Se descarta el formato binario porque el volumen de datos no justifica la complejidad adicional, el texto es más sencillo de diseñar y depurar.

3. Servidores — Hilos y Estado

3.1 Número de servidores

El sistema cuenta con dos tipos de servidor:

- **Servidor central:** escucha permanentemente en un puerto fijo. Recibe los mensajes OPEN, SALE y CLOSE provenientes de las cajas.
- **Servidor local en cada caja (x10):** cada caja tiene un hilo de escucha en un puerto fijo para poder responder a consultas STATUS del monitor, para poder conocer el estado de cada caja en cualquier momento.

3.2 Tipo de servidor: Iterativo vs Concurrente

La elección entre servidor iterativo y concurrente depende de si pueden llegar conexiones simultáneas entrantes o no. El **servidor central** puede recibir mensajes de varias cajas a la vez, por lo que si fuera iterativo obligaría a las cajas a esperar, así que debe ser **concurrente**. En el caso de los **servidores locales en cada caja** ocurre el caso contrario, como solo el monitor puede consultar dicha caja no hay concurrencia, por lo que puede ser **iterativo**.

3.3 Estado del servidor: Stateful vs Stateless

El **estado** hace referencia a la información que el servidor guarda en memoria entre mensajes sobre las cajas. Como es necesario que esta información sea almacenada, el servidor central debe ser **stateful**, manteniendo para cada caja datos como por ejemplo podrían ser id_caja, hora_apertura, etc.

Las cajas también son **stateful** ya que el hilo de escucha necesita conocer el estado actual de la caja para responder correctamente al monitor.

Resumen final de los servidores:

Entidad	Nº servidores	Tipo de hilos	Estado
Servidor central	1	Concurrente (1 hilo por caja)	Stateful
Servidor local de cada caja	1 por caja (10 servidores)	Iterativo	Stateful

4. Mensajes

Se definen cuatro tipos de mensaje, los cuales siguen todos el protocolo de texto que hemos explicado anteriormente, con campos separados por '|' y terminados en '\n'. Además se incluye un mensaje de confirmación **ACK** que el servidor central envía tras recibir correctamente cada mensaje.

Mensaje	Campos
OPEN (apertura de caja)	OPEN id_cajero num_caja hora_apertura

SALE (venta)	SALE id_cajero num_caja hora_venta importe_total n_articulos id_prod_1 importe_1 ... id_prod_n importe_n
CLOSE (cierre de caja)	CLOSE num_caja total_articulos minutos_abierta total_facturado
STATUS_REQ (petición monitor)	STATUS_REQ num_caja
STATUS_RES (caja cerrada) (respuesta caja)	STATUS_RES num_caja CLOSED
STATUS_RES (caja abierta) (respuesta caja)	STATUS_RES num_caja OPEN id_cajero minutos_abierta total_articulos total_facturado
ACK (confirmación servidor)	ACK OK ó ACK ERROR

5. Flujo de Mensajes

A continuación se muestran los diagramas de secuencia para cada caso de uso:

- Cajero abre → envía OPEN al central → central confirma (ACK).
- Se realiza venta → envía SALE → central confirma.
- Cajero cierra → envía CLOSE → central confirma.
- Monitor consulta → envía STATUS_REQ a la caja → recibe STATUS_RES.

Unos ejemplos de cómo sería el flujo real entre los mensajes serían:

Apertura, venta y cierre (Caja → Servidor central)

Textproto

Caja 5 → Servidor Central

```
|--- OPEN|cajero_01|5|08:30 -->| (registra apertura)
|<----- ACK|OK -----|
|--- SALE|...|52.30|2|... --->| (acumula venta)
|<----- ACK|OK -----|
|--- CLOSE|5|47|480|312.50 -->| (registra cierre)
|<----- ACK|OK -----|
```

Consulta monitorización (Monitor → Caja)

Textproto

```
Monitor (central) → Caja 5 (hilo de escucha)
|----- STATUS_REQ|5 ----->|
|<-- STATUS_RES|5|OPEN|cajero_01|120|23|187.50 ---|
```

6. Nombrado (IP y Puertos)

Se utiliza una red privada con el rango **10.0.0.0/24** (por ejemplo). Todas las IPs y puertos son estáticos y conocidos por todos los nodos en la red. Las cajas se conectan al servidor central como clientes utilizando un puerto efímero asignado automáticamente por el sistema operativo, por lo que no necesitan puerto fijo de origen (no se incluyen por tanto en la tabla). El nombrado de las entidades es por tanto:

Entidad	IP	Puerto
Servidor central	10.0.0.2	5000
Caja 1	10.0.0.3	5001
Caja 2	10.0.0.4	5001
...	...	...
Caja 10	10.0.0.12	5001

Decidimos las IPs y puertos de esta manera por lo siguiente:

- **10.0.0.2** para el servidor central ya que la **10.0.0.1** por defecto suele ser ocupada por el router.
- **10.0.0.3-10.0.0.12** para las cajas para así ocupar todo el espacio entre ellas y el servidor central, y no dejar huecos que desperdicien tamaño del rango de IPs.
- Elegimos los puertos **5000** y **5001** simplemente por no estar en el rango de los puertos bien conocidos (<1024), pero podríamos haber elegido cualquier otro. Lo mismo ocurre con el rango **10.0.0.0/24**, elegido simplemente por ser un rango común y cómodo.

7. Seguridad

- **Autenticación:** el campo id_cajero debe validarse en la base de datos de empleados antes de aceptar cualquier mensaje.
- **Integridad:** TCP garantiza entrega ordenada y sin errores, sin necesidad de mecanismos adicionales.
- **Confidencialidad:** se recomienda TLS/SSL sobre TCP para cifrar la comunicación.
- **Control de acceso:** el monitor debería requerir autenticación para evitar que cualquier equipo de la red pueda consultar el estado de las cajas.